Memory Management and Garbage Collection Strategies in High-Level Quantum Programming Languages

Rajveer Chopra
School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology
Vellore, Tamil Nadu, India
rajveerchopra55@gmail.com

***Abstract*—High-level quantum programming languages let programmers allocate temporary qubits as freely as local variables, but they leave the compiler a memory-management problem with no classical counterpart. By the no-cloning theorem a live qubit can be neither copied, nor inspected, nor silently dropped, so neither copying nor tracing garbage collection applies. This paper presents QRECLAIM, a compile-time garbage collector for qubits, built as a pass pipeline over a linear, gate-level intermediate representation (QR-IR). A linear type system rejects programs that would require cloning. A forward basis-state abstract interpretation computes, for every ancilla, its reversible computation trace, its dependency set and an uncomputation deadline, and an uncomputation-aware extension of live-variable analysis places every uncomputation at the earliest provably safe point. The same framework reclaims measured qubits by early measurement and reset, deletes ancillas that are never read (dead-garbage elimination), reorders gates to lower qubit pressure, and assigns physical qubits with an earliest-ready-first linear-scan allocator that exposes the width–depth trade-off as a single qubit budget. On five benchmark families (24 instances, up to 262 qubits), QRECLAIM reduces the number of physical qubits by 55.6% on average (up to 98.4%) relative to compilation without reuse, and by 42.3% relative to lexical, scope-based reclamation; its depth-neutral mode still saves 36.2% of the qubits with no increase in circuit depth. Semantic preservation was validated by exact simulation of 29 compiled programs with 609 run-time reclamation assertions, and compilation time grows linearly with program size.**

***Index Terms*—quantum compilers, uncomputation, qubit reuse, garbage collection, data-flow analysis, register allocation, no-cloning theorem**

I. INTRODUCTION

Current quantum processors offer tens to a few hundred noisy qubits, and fault-tolerant machines will spend most of their physical qubits on error correction. The number of logical qubits a program needs, its *width*, therefore decides whether it can run at all, and reducing it is one of the most valuable things a quantum compiler can do. At the same time, quantum programming is moving away from hand-built circuits towards high-level languages such as Silq [1], Qrisp [2] and Qurts [3], in which the programmer allocates temporary qubits (*ancillas*) the way a classical programmer declares local variables and expects the system to take them back. Returning qubits safely and early is the quantum analogue of memory management, and it is the subject of this case study.

Classical garbage collection does not carry over. A copying collector moves live objects, but a quantum state cannot be copied [4]. A tracing collector inspects the heap at run time, but measuring a qubit disturbs it. Most importantly, a dead temporary is usually *entangled* with live data. Discarding it decoheres the rest of the program and destroys the interference on which quantum algorithms depend. The only clean way to free such a qubit is to *uncompute* it by running the gates that computed it in reverse [5], which is valid only while the qubits it was computed from are unchanged. Mid-circuit measurement and reset offer a second mechanism for qubits whose value is no longer needed coherently [6], [7]. Memory management for qubits must therefore be done statically, by a compiler that understands both the program's data flow and its quantum semantics.

This raises five compiler design challenges:

1) *Safety (C1):* a qubit may be reclaimed only if this neither requires a copy nor leaves garbage entangled with live state.
2) *Placement (C2):* an uncomputation must happen after the ancilla's last use but before any of its inputs is overwritten, and ancillas computed from other ancillas extend each other's lifetimes.
3) *Unification (C3):* uncomputation-based and measurement-based reclamation should be decided in one framework rather than by separate tools.
4) *Allocation trade-off (C4):* reusing a qubit serialises computations that could otherwise run in parallel (a false dependency, exactly as in register allocation), so fewer qubits can mean deeper circuits.
5) *Cost (C5):* the analyses must be fast enough to run on every compilation.

This paper addresses these challenges with QRECLAIM, whose contributions are:

- **QR-IR and a linear typing discipline** (§III-B–III-C) that enforces no-cloning statically and turns qubit reclamation into a type-preserving program transformation.

- **A basis-state abstract interpretation and an uncomputation-aware liveness analysis** (§III-D), written as data-flow equations, with a legality condition whose soundness is argued in Theorem 1.
- **A unified reclamation pass** (§III-E) covering uncomputation, early-measure-and-reset, dead-garbage elimination (DGE), and hierarchical reclamation of the helper qubits introduced when multi-controlled gates are lowered.
- **Pressure-aware scheduling and earliest-ready-first (ERF) linear-scan allocation** (§III-F) with an explicit qubit budget, including a *depth-neutral* mode.
- **A working prototype and an evaluation** (§IV–V) on five program families, with every number regenerated by one script and every small instance verified by exact simulation.

II. Literature Review

A. High-level languages and qubit lifetimes

Silq [1] introduced *safe automatic uncomputation*. Annotations such as `qfree` and `const` identify computations that can be reversed and inputs that stay unchanged, temporaries are uncomputed implicitly, and the type system rejects programs whose uncomputation would be unphysical. Qurts [3] carries Rust-style affine types with lifetimes over to quantum programs. Inside its lifetime a qubit may be discarded, and is then uncomputed automatically; outside it, it must be used linearly. Qrisp [2] is a Python-embedded framework that compiles high-level programs down to circuits and supports automatic uncomputation and the reuse of deallocated qubits. These systems decide *whether* a value can be uncomputed. The point at which it is reclaimed, however, follows the lexical structure of the program (a scope, a lifetime or a function exit).

B. Uncomputation synthesis and pebbling

Bennett's compute–copy–uncompute construction [5] is the basis of all uncomputation. Unqomp [8] synthesises uncomputation automatically for the ancillas a programmer marks, working on a circuit-graph representation. Reqomp [9] adds a space constraint and trades recomputation for ancillas, reducing their number by up to 96%. SQUARE [10] decides at each modular call site whether to uncompute and reuse ancillas or keep the garbage, using a cost model that accounts for locality and parallelism. Meuli *et al.* [11] cast qubit clean-up as a reversible pebbling game solved with SAT. Liu *et al.* [12] recently unified the uncomputation of clean and dirty ancillas and proved that deciding whether an uncomputation exists is coNP-hard. That result justifies the conservative, efficiently checkable analysis adopted here.

C. Qubit reuse with dynamic circuits

Mid-circuit measurement and reset allow a qubit to be reused once its final measurement has been taken. CaQR [6] builds a compiler pass around this capability and reports resource reductions of up to 80%. DeCross *et al.* [7] formulate qubit-reuse compilation exactly as a constraint program, give a greedy heuristic, and run an 80-qubit QAOA circuit on a 20-qubit trapped-ion device.

TABLE I
Qubit-reclamation mechanisms in related work

Work	Mechanism	Decision basis
Silq [1]	implicit uncomputation	type annotations; lexical scope
Qurts [3]	implicit uncomputation	affine types with lifetimes
Qrisp [2]	uncomputation, recycling	high-level framework, Python-embedded
Unqomp [8]	uncomputation synthesis	circuit graph
Reqomp [9]	uncompute + recompute	circuit graph, ancilla budget
SQUARE [10]	uncompute or keep	cost model at call sites
Meuli [11]	clean-up strategy	reversible pebbling (SAT)
CaQR [6]	measure + reset	circuit DAG, reuse pairs
DeCross [7]	measure + reset	constraint program / greedy
QReclaim	uncompute, measure + reset, DGE	data-flow window, linear scan with budget

D. Quantum IRs and classical compiler techniques

QIRO [13] gives quantum programs an SSA-based representation with value semantics, so that standard data-flow optimisations apply. McCaskey and Nguyen [14] define an MLIR dialect that lowers quantum assembly to LLVM IR. On the classical side, linear-scan register allocation [15] assigns registers to live intervals in a single pass. QReclaim transfers liveness analysis and linear scan to qubits. Two features have no classical counterpart: *spilling* is impossible because there is no copy, and *freeing* requires uncomputation or measurement.

E. Identified research gaps

Table I summarises the mechanisms. Four gaps motivate this work:

- **G1** Uncomputation-based reuse [8]–[10] and measurement-based reuse [6], [7] are developed as separate techniques at different levels of abstraction. A program that needs both gets no unified treatment.
- **G2** Reclamation legality is stated as type rules [1], [3] or circuit-graph conditions [8] rather than as data-flow facts that compose with the liveness, scheduling and allocation passes of a conventional compiler.
- **G3** Reuse creates false dependencies. Apart from space-constrained [9] and cost-model-driven [10] approaches, the width–depth trade-off is rarely exposed as an explicit, user-visible knob, and simple qubit managers reuse in stack (LIFO) order.
- **G4** Where reclamation is lexical, its quality depends on how the programmer happens to nest scopes rather than on the program's actual data flow.

III. Proposed Model / Architecture

A. System overview

Fig. 1 shows the compilation pipeline. Programs are written in an embedded front end and translated to QR-IR, a linear list of instructions over *logical* qubits. A linear type check enforces no-cloning (§III-C). Two analyses then run over the high-level IR: a forward *basis-state abstract interpretation* and

a backward *liveness* analysis (§III-D). The *level-1 reclamation pass* uses their results to insert uncomputations, early measurements, resets and `free` instructions for programmer ancillas (§III-E). Multi-controlled gates are then lowered to Toffoli chains, and a *level-2 reclamation pass* reclaims the helper qubits this creates. Finally, a pressure-aware scheduler reorders commuting instructions, an ERF linear-scan allocator maps logical to physical qubits under an optional budget (§III-F), and code is emitted as a Qiskit circuit or OpenQASM 2. The type check runs again after scheduling, and a physical no-aliasing check after allocation.

B. Intermediate representation (QR-IR)

A QR-IR program is a sequence $P = \iota_0 \dots \iota_{N-1}$ of

$$\begin{aligned}\iota ::= {} & \texttt{alloc}\ q \mid \texttt{free}\ q \mid g^{\theta}(\bar{c}^{\bar{\pi}}; \bar{t}) \\ & \mid \texttt{measure}\ q \to c \mid \texttt{reset}\ q \mid \texttt{end}\ s\end{aligned}$$

where $\bar{c}$ are control qubits with polarities $\pi \in \{+, -\}$ (a $-$ control fires on $|0\rangle$), $\bar{t}$ are targets, and θ are optional angles. CX, Toffoli and k-controlled X are all the base operation `x` with $|\bar{c}| = 1, 2, k$, which keeps the analyses uniform. Each qubit has a kind: *data* (owned by the programmer), *ancilla* (reclaimed by the compiler) or *helper* (created by lowering). Base gates fall into three classes according to how they act on computational-basis states:

- $G_{\mathrm{qf}} = \{\texttt{x}, \texttt{swap}\}$ *permute* the basis (qfree in Silq's terminology [1]);
- $G_{\mathrm{dg}} = \{\texttt{z}, \texttt{s}, \texttt{s}^{\dagger}, \texttt{t}, \texttt{t}^{\dagger}, \texttt{rz}, \texttt{p}\}$ are *diagonal*: they only add phases;
- $G_{\mathrm{gen}} = \{\texttt{h}, \texttt{rx}, \texttt{ry}\}$ can create superposition.

The *write set* $W(\iota)$ contains the targets of $G_{\mathrm{qf}} \cup G_{\mathrm{gen}}$ gates and the target of `reset`. These are exactly the instructions that can change a qubit's basis value, and controls and diagonal gates never write. Negative controls are kept symbolic until code generation, so the X conjugation that implements them is never mistaken for a write.

The embedded front end (Python) offers `qreg`, `ancilla`, gate methods and a `scope` context manager. Programmers write only the *forward* computation and never uncompute or free anything by hand (Fig. 2a).

C. Static semantics: linear typing enforces no-cloning

Typing judgements $\Delta \vdash \iota \dashv \Delta'$ thread a context Δ that maps each live qubit to its abstract state (§III-D1):

$$\frac{q \notin \mathrm{dom}(\Delta)}{\Delta \vdash \texttt{alloc}\ q \dashv \Delta, q{:}\mathbf{0}}\ (\textsc{T-Alloc})$$

$$\frac{\Delta(q) = \mathbf{0}}{\Delta \vdash \texttt{free}\ q \dashv \Delta \setminus q}\ (\textsc{T-Free})$$

$$\frac{\bar{c}, \bar{t}\ \text{pairwise distinct} \qquad \{\bar{c}, \bar{t}\} \subseteq \mathrm{dom}(\Delta)}{\Delta \vdash g(\bar{c}; \bar{t}) \dashv \Delta[\bar{t} \mapsto \tau_g(\Delta, \bar{c}, \bar{t})]}\ (\textsc{T-Gate})$$

No rule duplicates a binding, and T-Gate requires distinct operands. A well-typed program can therefore never use two "copies" of one qubit, which is the static form of the no-cloning theorem [4]. Discharging T-Free requires a proof that $\Delta(q) = \mathbf{0}$, and such proofs are produced only by the reclamation pass, through Theorem 1 or Lemma 1. The checker runs on the logical program and again after scheduling. After allocation, a physical variant verifies that no physical qubit ever hosts two live logical qubits.

TABLE II
TRANSFER FUNCTIONS OF THE BASIS-STATE ANALYSIS (INSTRUCTION ι AT POSITION i)

Instruction ι	Effect on the abstract facts
`alloc` q	$\sigma(q) \leftarrow \mathbf{0}$; $Tr(q) \leftarrow \epsilon$; $Dep(q) \leftarrow \emptyset$; $D(q) \leftarrow \infty$
$g(\bar{c}; t)$, $g \in G_{\mathrm{qf}}$, if $\sigma(t) \in \{\mathbf{0}, \mathbf{B}\}$ and $D(t) = \infty$	$\sigma(t) \leftarrow \mathbf{B}$; $Tr(t) \leftarrow Tr(t) \cdot \iota$; $Dep(t) \leftarrow Dep(t) \cup \bar{c}$
otherwise, or $g \in G_{\mathrm{gen}}$	$\sigma(t) \leftarrow \mathbf{T}$ for every target t
$g \in G_{\mathrm{dg}}$	no change (adds phases only)
`measure` q	$\sigma(q) \leftarrow \mathbf{M}$
`reset` q	$\sigma(q) \leftarrow \mathbf{0}$; $Tr(q) \leftarrow \epsilon$; $Dep(q) \leftarrow \emptyset$
any ι, each $w \in W(\iota)$	for all a with $w \in Dep(a)$ and $D(a) = \infty$: $D(a) \leftarrow i$

D. Data-flow analyses

1) Basis-state abstract interpretation (forward): Each qubit q carries an abstract state $\sigma(q)$ from the lattice

$$\mathbf{0} \sqsubseteq \mathbf{B} \sqsubseteq \mathbf{T}, \qquad \mathbf{M} \sqsubseteq \mathbf{T},$$

where $\mathbf{0}$ means exactly $|0\rangle$, $\mathbf{B}$ means a basis value that is a reversible function of other qubits, $\mathbf{M}$ means measured (collapsed), and $\mathbf{T}$ means possibly in superposition. It also carries a *trace* $Tr(q)$ (the qfree gates that computed it), a *dependency set* $Dep(q)$ (the controls those gates read) and a *deadline* $D(q)$: the position of the first later instruction that writes some $d \in Dep(q)$. Table II gives the transfer functions. The analysis is a single forward sweep, because after inlining and unrolling a quantum program is one basic block. Each write clears the reverse map from the written qubit to its dependants, so the deadline bookkeeping costs amortised $O(1)$ per instruction.

2) Liveness (backward): With $DEF(\iota) = \{q\}$ for `alloc` q and $USE(\iota)$ the operands of every other instruction, the classical live-variable equations apply unchanged:

$$LIVE_{in}(\iota) = USE(\iota) \cup \big(LIVE_{out}(\iota) \setminus DEF(\iota)\big), \tag{1}$$

$$LIVE_{out}(\iota) = \bigcup_{s \in succ(\iota)} LIVE_{in}(s), \tag{2}$$

$$L(q) = \max\{\, i : q \in USE(\iota_i) \,\}. \tag{3}$$

On straight-line code a single backward sweep solves (1)–(2) and yields the last use $L(q)$ and the qubit pressure $|LIVE|$ at every point.

3) Uncomputation-aware liveness and placement: Inserting the uncomputation $U(a)$ of an ancilla a at position $P(a)$ *reads* every $d \in Dep(a)$ again. The last use of a dependency is

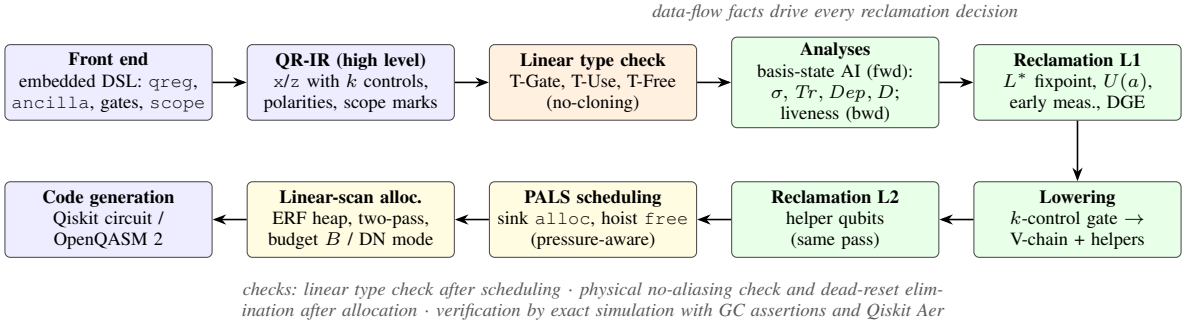


Fig. 1. Architecture of the QRECLAIM compilation pipeline. Green stages form the compile-time garbage collector; the same reclamation pass runs at two IR levels.

therefore not $L(d)$ but may be later. Let R be the set of ancillas to uncompute. The effective last use is the least fixpoint of

$$L^*(q) = \max\big(L(q),\ \max\{P(a) : a \in R,\ q \in Dep(a)\}\big), \tag{4}$$

$$P_{\text{eager}}(a) = L^*(a), \tag{5}$$

$$P_{\text{lazy}}(a) = \min\big(end(scope(a)),\ D(a){-}1, \min\{P_{\text{lazy}}(b) : b \in Dep(a) \cap R\}\big), \tag{6}$$

and a placement is *legal* iff

$$L^*(a) \ \le\ P(a) \ <\ D(a) \qquad \text{for every } a \in R. \tag{7}$$

Equation (5) is QRECLAIM's policy: reclaim as early as possible. Equation (6) models lexical reclamation and serves as the SCOPE baseline. It is deliberately *stronger* than a strict scope rule, because it moves an uncomputation before a deadline that falls inside the scope instead of rejecting the program. If (7) fails for some a, the program is rejected with a diagnostic naming the instruction that overwrote a's input (§V-G).

The fixpoint is monotone over finite positions, so it terminates. Processing R in reverse order of first computation follows a reverse topological order of the dependency DAG, and the fixpoint then usually stabilises after one sweep.

Theorem 1 (Sound uncomputation): Let $\sigma(a) = \mathbf{B}$ at the end of P and let $U(a) = Tr(a)_k^{-1} \cdots Tr(a)_1^{-1}$. Inserting $U(a)$; `free` a after any position p with $L^*(a) \le p < D(a)$ leaves a in $|0\rangle$, unentangled from the other qubits, and yields the same final state on all other qubits for every such p.

Proof sketch: Expand the state in the computational basis of $Dep(a)$. Because $p < D(a)$, no instruction between the first trace gate and p writes a dependency, so in each branch $|d\rangle$ the controls of every trace gate hold the same values when the gate executes and when it is reversed. Every other instruction touching a only *reads* it, as a control or through a diagonal gate; a write would have made $\sigma(a) = \mathbf{T}$. So in branch $|d\rangle$ the ancilla holds $f(d)$ from the end of its trace onwards, and $U(a)$ maps $|d\rangle|f(d)\rangle \mapsto |d\rangle|0\rangle$. By linearity the global state factorises as $|\psi\rangle \otimes |0\rangle_a$. For position independence, note that an instruction inside the window does not touch a (because $p \ge L^*(a) \ge L(a)$) and touches $Dep(a)$ only as controls, through diagonal gates or by measurement (because $p < D(a)$). Such an instruction and $U(a)$ are both block-diagonal in the computational basis of $Dep(a)$ and act on disjoint qubits within every block, so they commute. ∎

Lemma 1 (Early measurement): If the final operation on q is `measure` $q \to c$ at position m and the previous use of q is at $\ell < m$, moving the measurement to just after $\max(\ell, \max\{P(b) : q \in Dep(b)\})$ and following it with `reset` q; `free` q preserves the joint distribution of all classical outcomes and the state of all other qubits.

Proof sketch: Between the new and the old position no instruction acts on q except controlled operations of uncomputations that only read it. Measurement commutes with operations on disjoint qubits and with operations that use q only as a control, so the outcome distribution is unchanged. The reset, a write, is ordered after every uncomputation that reads q. ∎

E. Reclamation transformations

Algorithm 1 summarises the pass. It is parameterised by the qubit kinds it manages, so the same code runs at both IR levels.

Dead-garbage elimination (DGE). An ancilla in state $\mathbf{0}$ or $\mathbf{B}$ whose only uses are its own trace gates is never read. Its computation and its uncomputation would cancel, so both are deleted together with the allocation. This removes, for example, the unused carry-out of an adder.

Uncomputation insertion. For every $a \in R$, the pass inserts $U(a)$ followed by `free` a after $\iota_{P(a)}$. The gates of $U(a)$ are the inverses of the trace in reverse order; for G_{qf} each gate is its own inverse.

Measurement-based reclamation. Lemma 1 is applied to every qubit (data or ancilla) whose final operation is a measurement.

Ordering. Several blocks may share an insertion point. A block that reads q must precede the block that writes q; ties are broken in reverse allocation order, in the Bennett style. This is a topological sort over at most a handful of blocks.

Diagnostics. An ancilla that ends in $\mathbf{T}$ and is never measured cannot be reclaimed without leaving entangled garbage. It is reported statically instead of being silently leaked.

Algorithm 1 RECLAIM$(P, K, mode)$

Require: program $P = \iota_0 \dots \iota_{N-1}$, managed kinds K
$(\sigma, Tr, Dep, D) \leftarrow$ BASISAI(P); $L \leftarrow$ LIVENESS(P)
$G \leftarrow \{a \in K : \sigma(a) \in \{\mathbf{0}, \mathbf{B}\} \wedge uses(a) = Tr(a)\}$ ▷ DGE
delete $Tr(a)$ and `alloc` a for every $a \in G$
if $\exists a \in K \setminus G : \sigma(a) = \mathbf{T}$ **then error** "unreclaimable ancilla"
end if
$R \leftarrow \{a \in K \setminus G : \sigma(a) = \mathbf{B}\}$
if $mode =$ EAGER **then**
  repeat
    for $a \in R$ in reverse order of first trace gate **do**
      $P(a) \leftarrow \max(L(a), \max\{P(b) : b \in R,\ a \in Dep(b)\})$
    end for
  until no $P(a)$ changed
else $P(a) \leftarrow$ Eq. (6) by memoised recursion
end if
for all $a \in R$ **do check** Eq. (7) **else error**
end for
for all q whose final operation is `measure` at m **do**
  $p \leftarrow \max(\max(uses(q) \setminus \{m\}),\ \max\{P(b) : q \in Dep(b)\})$
  move ι_m after ι_p; append `reset` q; `free` q
end for
for all $a \in R$ **do** insert $U(a)$; `free` a after $\iota_{P(a)}$
end for
order blocks at equal points: readers of q before writers of q

Algorithm 2 ALLOCATE(P, B) (ERF linear scan)

$heap \leftarrow \emptyset$; $ready \leftarrow [\,]$
for all $\iota \in P$ in order **do**
  if $\iota =$ `alloc` q **then**
    if $(B \neq \perp \wedge |ready| \geq B) \vee (B = \perp \wedge heap \neq \emptyset)$ **then**
      $(_, p) \leftarrow$ POPMIN$(heap)$ ▷ earliest ready
    else $p \leftarrow |ready|$; append 0 to $ready$ ▷ fresh
    end if
    $\phi(q) \leftarrow p$
  else if $\iota =$ `free` q **then**
    PUSH$(heap, (ready[\phi(q)], \phi(q)))$
  else
    $t \leftarrow 1 + \max\{ready[\phi(q)] : q \in ops(\iota)\}$
    $ready[\phi(q)] \leftarrow t$ for every $q \in ops(\iota)$
  end if
end for
return ϕ ▷ two-pass: $B \leftarrow$ width of ALLOCATE$(P, \perp)$, rerun

Hierarchical reclamation. A gate with $k \geq 3$ controls is lowered to a Barenco-style V-chain of $k-2$ helper qubits followed by one Toffoli or CCZ on the target. Lowering emits only the *compute* half of the chain; the uncompute half is produced by running Algorithm 1 again with $K = \{\text{helper}\}$. Running level 1 *before* lowering is essential. At level 1 a clause ancilla computed by a 3-controlled gate depends only on the three input qubits. After lowering it would also depend on the helper that carries the conjunction, and that dependency would keep the helper alive until the clause itself is uncomputed.

F. Scheduling and allocation

Pressure-aware list scheduling (PALS). Gates on disjoint qubits commute, so any topological order of the per-qubit precedence DAG is equivalent to the input. PALS emits ready instructions with priority `free` $\prec$ (gates, `measure`, `reset` in source order) $\prec$ `alloc`, and orders `alloc`s by their qubit's first use. This sinks allocations and hoists reclamation, the program-level counterpart of the reuse pairs found by CaQR [6]. Bernstein–Vazirani, for example, turns into the qubit-sequential form that needs only two qubits.

Earliest-ready-first linear scan. After scheduling, every logical qubit occupies one interval $[\texttt{alloc}, \texttt{free}]$, so the interference graph is an interval graph.

Proposition 1: For a fixed instruction order, let $\omega = \max_i |A(i)|$, where $A(i)$ is the set of logical qubits allocated but not yet freed at position i. A linear-scan allocator that reuses a free physical qubit whenever one exists uses exactly ω physical qubits, which is the minimum possible. A second pass with budget $B = \omega$ that allocates fresh qubits first never fails.

Proof sketch: Interval graphs are perfect, and colouring intervals greedily in order of their left endpoints (the order in which linear scan [15] visits them) uses exactly ω colours. In the second pass, at most $\omega - 1$ other logical qubits are live at any allocation, so once all ω physical qubits exist at least one of them is free. ∎

Which free qubit to choose decides the depth cost of a reuse. Our allocator tracks the ASAP finishing time $ready[p]$ of every physical qubit. When it reuses, it takes the pooled qubit with the smallest $ready$ value (earliest-ready-first, ERF), which minimises the false dependency the reuse introduces. A pooled qubit executes no operations, so its key cannot change while it waits, and a binary heap gives $O(\log n)$ selection (Algorithm 2).

Budget and depth-neutral mode. With a budget B the allocator uses fresh qubits until B exist and only then reuses. Two settings are of interest. QRECLAIM-*min* uses $B = \omega$ (Proposition 1). QRECLAIM-*DN* (depth-neutral) bisects for the smallest B whose circuit is no deeper than the no-reuse circuit. Afterwards, *dead-reset elimination* removes resets on physical qubits that are never handed out again.

Complexity. With N instructions of bounded arity, the analyses and the type checks are $O(N)$, the placement fixpoint is $O(h \cdot \sum_a |Dep(a)|)$ for a dependency DAG of height h (one sweep in practice), PALS and ERF allocation are $O(N \log N)$, and DN mode adds a factor of $\log_2 W_{\mathrm{NR}}$.

G. Worked example

Fig. 2 compiles a two-term inner-product phase oracle. Under lexical reclamation both temporaries live until the end of the oracle's scope, which takes 6 qubits. QRECLAIM uncomputes t_0 immediately after its last use, hoists the final `h` and `measure` of x_0, y_0, and hands t_0's physical qubit to t_1, which takes 5 qubits. At the scale of the HS benchmark this effect turns $3n$ qubits into $2n{+}1$.

IV. Implementation / Experimental Setup

A. Toolchain

QRECLAIM is implemented in about 1,400 lines of Python 3.13. There is one module per stage of Fig. 1: the IR and front end, the type checker, the analyses, reclamation, lowering, scheduling, allocation, code generation, a simulator and the driver. Code generation targets Qiskit 2.5.2, which also lowers every compiled circuit to the $\{\mathrm{CX}, U\}$ basis at optimisation level 1 so that gate counts and depths are

(a) source (embedded DSL)

```
x = p.qreg("x", 2); y = p.qreg("y", 2)
for q in x + y: p.h(q)
with p.scope():                    # oracle (-1)^{x.y}
    for i in range(2):
        t = p.ancilla("t")         # never freed by hand
        p.ccx(x[i], y[i], t); p.z(t)
for q in x + y: p.h(q)
for q in x + y: p.measure(q)
```

(b) SCOPE: 6 qubits

```
alloc x0..y1      ; q0–q3
h x0; h x1; h y0; h y1
alloc t0          ; q4
ccx x0,y0 -> t0
z t0
alloc t1          ; q5
ccx x1,y1 -> t1
z t1
ccx x1,y1 -> t1   ; U(t1)
free t1
ccx x0,y0 -> t0   ; U(t0)
free t0
h x0; h x1; h y0; h y1
measure x0, x1, y0, y1
```

(c) QRECLAIM: 5 qubits

```
alloc x0..y1; h ...  ; q0–q3
alloc t0             ; q4
ccx x0,y0 -> t0
z t0
ccx x0,y0 -> t0      ; U(t0)
free t0              ; q4 free
h x0; measure x0     ; hoisted
h y0; measure y0     ; hoisted
alloc t1             ; q4 again
ccx x1,y1 -> t1
z t1
ccx x1,y1 -> t1      ; U(t1)
free t1
h x1; measure x1
h y1; measure y1
```

Fig. 2. Worked example. (a) Source code with two implicit temporaries. (b) Lexical reclamation keeps both until the end of the scope. (c) QRECLAIM reclaims t_0 at its last use and reuses its qubit for t_1. Listings (b) and (c) are actual compiler output, condensed.

reported on a common footing. Programs can be exported as OpenQASM 2, including mid-circuit `measure` and `reset`. Correctness is checked with a purpose-built exact simulator (NumPy, branching on mid-circuit measurements, so the output distribution is exact rather than sampled) and cross-checked with Qiskit Aer 0.17.2 at 8,192 shots. The simulator asserts on every `alloc` and `free` that the qubit is exactly $|0\rangle$, which checks at run time what reclamation guarantees statically. All experiments ran single-threaded on an Intel Xeon processor at 2.10 GHz (4 vCPUs, Linux), and one script (`experiments/run_all.py`) regenerates every table and figure. A pytest suite of 99 test cases covers all configurations and benchmarks.

B. Benchmarks

The five families in Table III are written in the high-level style the paper targets: temporaries are allocated freely and never uncomputed by hand, and each family stresses a different reclamation mechanism.

C. Configurations and metrics

Four configurations are compared, all running through the same front end, lowering and code generator:

- **NR** (no reuse): lazy placement and a fresh physical qubit for every allocation, i.e. correct uncomputation without any qubit reuse.
- **SCOPE**: lazy placement (Eq. 6) with stack (LIFO) reuse of reclaimed ancillas, an idealised model of lexical reclamation.
- **QRECLAIM-min**: eager placement, early measurement, DGE, PALS and two-pass ERF allocation.
- **QRECLAIM-DN**: the same pipeline under the smallest depth-neutral budget.

TABLE III
BENCHMARK FAMILIES (24 INSTANCES IN TOTAL)

ID	Program	Sizes	Stresses
BV	Bernstein–Vazirani, n-bit secret	n=8..128	measured data
HS	hidden shift, inner-product bent function on $2n$ bits; one temporary per product	n=4..64	flat temporaries
RC	repetition-code memory, distance d, r=d rounds, fresh syndrome qubit per check	d=3..11	mid-circuit measurement
ADD	ripple-carry adder, carries in ancillas, unused carry-out	n=4..64	dependency chains, DGE
GRV	Grover search, planted 3-SAT, n variables, m=$2n$ clauses, one ancilla per clause	n=4..10	iterated oracles, lowering

TABLE IV
VERIFICATION: WIDTH PER CONFIGURATION, RECLAMATION ASSERTIONS AND TVD OF QRECLAIM-MIN FROM THE EXPECTED DISTRIBUTION

	width			GC	TVD	TVD
Instance	NR	SCOPE	QRECLAIM	checks	exact	Aer
BV (n=8)	9	9	2	16	3.3e-16	0.000
BV (n=12)	13	13	2	24	1.3e-15	0.000
HS (n=3)	12	9	7	19	1.1e-15	0.000
HS (n=4)	16	12	9	26	2.1e-15	0.000
RC (d=3, r=3)	9	9	4	14	0.0e+00	0.000
RC (d=5, r=3)	17	17	6	28	0.0e+00	0.000
ADD (n=4)	12	12	11	14	0.0e+00	0.000
ADD (n=5)	15	15	14	18	0.0e+00	0.000
GRV (n=4, m=4)	18	9	9	32	1.5e-15	0.009
GRV (n=5, m=6)	28†	14	14	51	2.1e-15	0.020

†too wide for exact simulation; other configurations verified.

An ablation adds the components one at a time. Metrics are *width* (physical qubits, the memory footprint), *CX* and total $\{\mathrm{CX}, U\}$ gate count (code size), *depth* after lowering (a proxy for execution time), the number of mid-circuit *resets*, and *compile time* (wall clock, best of three).

V. RESULTS & DISCUSSION

A. Correctness

Table IV lists the verification instances. Every compiled program whose width allowed exact simulation (29 of 30) reproduced the analytically expected output distribution: the BV secret, the hidden shift, the syndrome pattern of the injected error, $(a + b) \bmod 2^n$, and the closed-form Grover distribution $\sin^2((2k{+}1)\theta)$. The total variation distance (TVD) is at most 2.7×10^{-15}, i.e. floating-point round-off. All 609 run-time reclamation assertions held. Qiskit Aer agrees within sampling noise: TVD 0 for the deterministic programs and at most 0.020 for Grover at 8,192 shots.

B. Memory footprint

Fig. 3 and Table V report width. Averaged over all 24 instances, QRECLAIM-min uses 55.6% fewer qubits than NR (geometric-mean width ratio 0.29, i.e. 3.4× fewer) and 42.3% fewer than SCOPE. SCOPE itself saves 14.9% over NR. The gains differ by mechanism:

- *BV and RC* need early measurement plus scheduling. Their width collapses to 2 and $d{+}1$ qubits independently of n and of the number of rounds (98.4% and 90.1% at the largest sizes). Lexical reclamation cannot reuse measured qubits at all, so SCOPE equals NR.
- *HS* profits from eager placement. All n temporaries share one lexical scope, so SCOPE needs $3n$ qubits; eager reclamation needs $2n{+}1$, 32.8% fewer than SCOPE and 49.6% fewer than NR.
- *GRV* profits from reuse across oracle calls, which SCOPE also exploits (82.1% fewer qubits than NR at $n{=}10$).
- *ADD* shows the limit of reclamation without recomputation. Carry c_{i+1} is computed from c_i, so by Eq. (4) $L^*(c_i) \geq P(c_{i+1})$, and the carries form a chain that eager and lazy placement handle identically. The only gain is DGE removing the unused carry-out (one qubit). Notably, eager placement independently rediscovers the interleaving of carry uncomputation and sum computation used in textbook ripple-carry adders.

C. Code size and depth

Reclamation itself adds no gates. Every configuration must uncompute its ancillas for correctness, so the uncomputation gates appear in NR as well, and the CX counts of QRECLAIM and NR are identical except where DGE fires (ADD: -20.6% CX at $n{=}4$, -1.0% at $n{=}64$). The extra operations of QRECLAIM-min are mid-circuit resets, one per reuse of a measured qubit. Width, however, is bought with depth. Reuse serialises work, and QRECLAIM-min is $3.1\times$ deeper than NR (geometric mean), up to $28.9\times$ for HS-64, whose single shared temporary forces all products to run one after another. The ERF heuristic matters here. On GRV, SCOPE's LIFO reuse inflates depth by 20–59%, while QRECLAIM-min reaches the *same* width with only 3–5% extra depth (Table VI).

D. Width–depth trade-off

Fig. 4 sweeps the budget B from the minimum width to the NR width. Depth falls steeply and reaches the NR depth at 33% (BV), 51% (GRV), 59% (RC) and 75% (HS) of the NR width, while the number of resets decreases linearly. QRECLAIM-DN selects this point automatically. With *zero* depth overhead on every instance it saves 36.2% of the qubits on average (90.7% on BV-128, 81.3% on GRV-10, 44.6% on RC-11). On GRV it is even marginally shallower than NR (-0.7%) and uses up to 36 fewer CX gates, because reuse places the closing x of one clause ancilla next to the opening x of the next on the same physical qubit, where Qiskit's peephole pass cancels them.

E. Ablation

Table VI adds the components one at a time. Each mechanism is responsible for a different family: eager placement for HS ($48 \to 33$ qubits), early measurement for RC ($49 \to 8$), PALS for BV ($33 \to 2$), DGE for ADD ($48 \to 47$, -22 CX), and the allocator for depth (ERF and two-pass reduce GRV's depth from 1094 to 825 and HS's from 671 to 371 at unchanged width). No single classical technique reproduces the combined effect, which supports gap G1.

F. Compilation time

All 24 main instances compile in at most 16 ms. Fig. 5 scales the inputs to 26,594 output instructions. A log–log fit of compile time against program size has slope 1.04, i.e. linear time at roughly $8\,\mu$s per instruction, and the full QRECLAIM pipeline is comparable to Qiskit's own level-1 transpilation of its output on HS, ADD and GRV (e.g. 206 ms vs. 238 ms for HS-1024). It is slower on BV, where Qiskit has little work per instruction. On the largest inputs the two reclamation passes account for 52–54% of the time, scheduling for 18–22%, allocation for 14–17% and type checking for 9–10%. The depth-neutral search costs at most 0.51 s, dominated by the Qiskit transpilation of each candidate.

G. Static diagnostics

The same facts that drive reclamation also catch programming errors at compile time. The three programs below are rejected with the messages shown, which are verbatim compiler output:

```
E1  p.cx(x0, x0)
    no-cloning violation at [1] `cx x0 -> x0`: qubit x0 is
     used twice by one operation
E2  t = x0 & x1; p.h(x0); p.cx(t, out)
    cannot uncompute ancilla t0: its control x0 is modified
     by `h x0` at [5] while t0 is still needed at [6]
E3  p.h(t); p.cx(t, out)
    ancilla t0 can never be reclaimed: `h t0` at [2] puts t0
      in superposition, and it is not measured
```

E2 is precisely a violation of Eq. (7). Silq reports comparable errors through its type system [1]; here they fall out of the data-flow analysis.

H. Discussion and threats to validity

- *Depth model.* Depth is measured after lowering to $\{\mathrm{CX}, U\}$ with all-to-all connectivity. Real devices add routing, and a mid-circuit reset is much slower than a gate on superconducting hardware, so QRECLAIM-min trades execution time for qubits. QRECLAIM-DN avoids this trade by construction.
- *Baseline.* SCOPE is our idealised model of lexical reclamation, implemented in the same framework and deliberately strengthened (Eq. 6). It is not a re-implementation of Silq, Q# or Qrisp, whose actual behaviour may differ.
- *Benchmarks.* The benchmarks are small to medium and were chosen to isolate mechanisms rather than to represent a workload distribution.
- *Verification.* Exact simulation is limited to about 20 qubits. Larger instances rely on the static guarantees, and the linear type check and the physical no-aliasing check run on every compilation.
- *Analysis precision.* The basis-state analysis is conservative: an ancilla touched by a non-qfree gate is never uncomputed by reversal. Given the coNP-hardness result of [12], some precision loss is unavoidable for an analysis that runs in linear time.

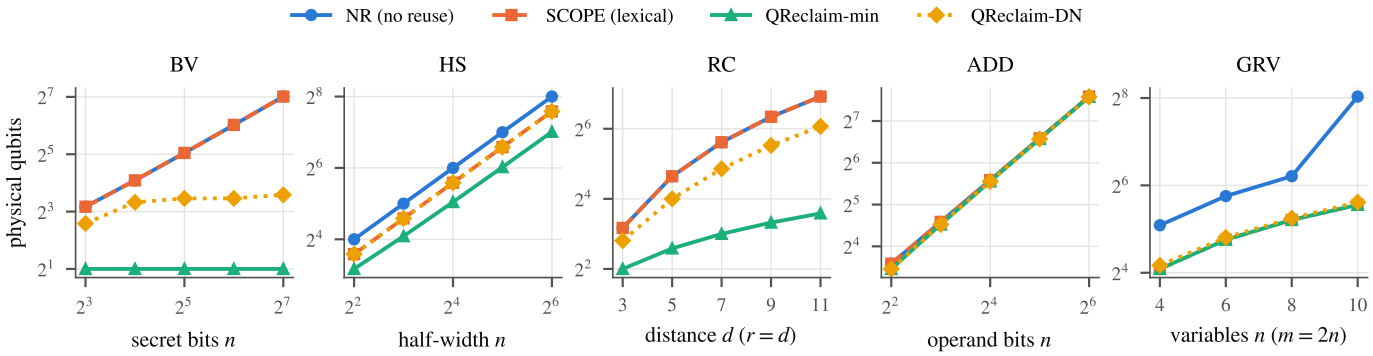


Fig. 3. Physical qubits (log scale) versus problem size. SCOPE coincides with NR on BV and RC because lexical reclamation cannot reuse measured qubits; QRECLAIM-DN keeps the depth of NR.

TABLE V
MAIN RESULTS AT THE LARGEST SIZE OF EACH FAMILY (WIDTH REDUCTION RELATIVE TO NR IN PARENTHESES)

	Width (physical qubits)				Depth				CX gates		Resets	Compile
Benchmark	NR	SCOPE	QRECLAIM-min	QRECLAIM-DN	NR	SCOPE	QRECLAIM-min	QRECLAIM-DN	NR	QRECLAIM-min	QRECLAIM-min	ms
BV (n=128)	129	129	**2** (98%)	12 (91%)	67	67	447	67	64	64	127	11.9
HS (n=64)	256	192	**129** (50%)	192 (25%)	44	44	1271	44	1536	1536	31	15.2
RC (d=11, r=11)	121	121	**12** (90%)	67 (45%)	42	42	405	42	220	220	109	5.8
ADD (n=64)	192	192	**191** (1%)	191 (1%)	2962	2962	2935	2935	2267	2245	0	11.7
GRV (n=10, m=20)	262	47	**47** (82%)	49 (81%)	2455	3899	2557	2437	3060	3060	0	9.6

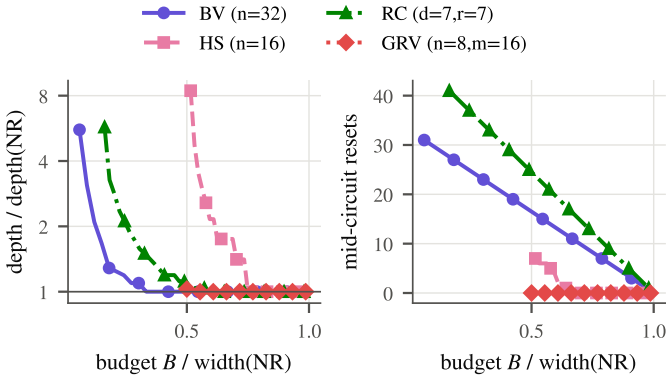


Fig. 4. Effect of the qubit budget B (normalised by the NR width). Left: depth relative to NR. Right: mid-circuit resets.

TABLE VI
ABLATION: WIDTH / DEPTH AS COMPONENTS ARE ADDED (SIZES AS IN FIG. 4; ADD AT n=16)

Configuration	BV-32	HS-16	RC-7	ADD-16	GRV-8
SCOPE	33 / 21	48 / 44	49 / 26	48 / 706	37 / 1094
+eager	33 / 21	33 / 581	49 / 26	48 / 706	37 / 1094
+early-meas	33 / 21	33 / 581	8 / 167	48 / 706	37 / 1094
+PALS	2 / 117	33 / 671	8 / 167	48 / 706	37 / 1094
+DGE	2 / 117	33 / 671	8 / 167	47 / 679	37 / 1094
+ERF	2 / 117	33 / 371	8 / 158	47 / 679	37 / 943
+two-pass	2 / 117	33 / 371	8 / 149	47 / 679	37 / 825

VI. CONCLUSION & FUTURE WORK

This case study treated qubit reclamation in high-level quantum languages as a compiler memory-management problem and solved it with standard compiler machinery adapted to quantum constraints. A linear IR enforces no-cloning, a basis-state abstract interpretation and an uncomputation-aware liveness analysis decide where reclamation is legal, a unified pass inserts uncomputation, early measurement and resets, and an earliest-ready-first linear-scan allocator exposes the width–depth trade-off through one budget. Across five program families, QRECLAIM reduced width by 55.6% on average (up to 98.4%), by 42.3% relative to lexical reclamation, and by 36.2% with no depth penalty, while compiling in linear time and passing every semantic check. Five extensions are technically meaningful:

1) **Recomputation-aware placement.** Chains such as ADD's carries block early reclamation, because $L^*(c_i) \geq P(c_{i+1})$. The next step is to extend Eq. (4) with optional *recomputation* edges: uncompute c_i early and recompute it just before $U(c_{i+1})$. Placement then becomes a min-cost reversible pebbling problem [9], [11] under budget B, solvable by ILP/SAT for small DAGs and by a fixpoint heuristic otherwise.
2) **Hardware-aware ERF allocation.** Replace unit latencies with device-calibrated gate, measurement and reset times and per-qubit error rates, and add coupling-map distance to the ERF key. This makes allocation a weighted interval-colouring problem that must be co-optimised with SWAP routing, because reuse changes the interaction graph the router sees.
3) **Dirty-ancilla borrowing.** Extend the lattice with a *borrowed* state so that lowering can use idle qubits that are

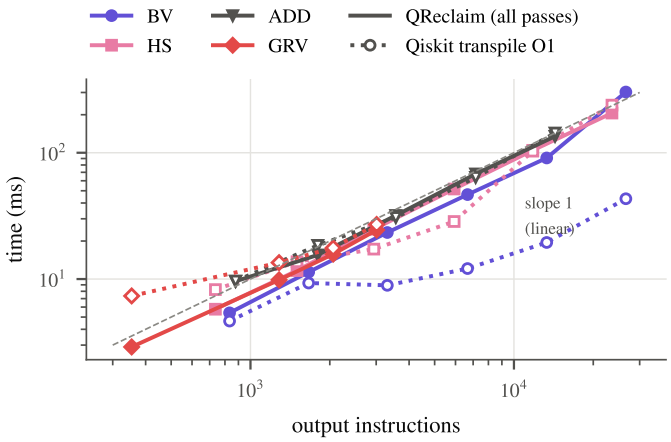


Fig. 5. Compile time versus program size (log–log): the QRECLAIM pipeline (solid) and Qiskit level-1 transpilation of the same output (dotted).

not clean, provided they are restored afterwards. Proving restoration is coNP-hard in general [12], which calls for sound syntactic templates backed by SMT checks.

4) **Control flow and feed-forward.** Lift the analyses from one basic block to CFGs with classically controlled branches (dynamic circuits). This requires a join on the abstract-state lattice, liveness across branches and region-based reclamation for repeat-until-success loops.
5) **Learned budget selection.** Predict the depth-neutral knee of Fig. 4 from IR features such as DAG width, dependency-DAG height and measurement density. A learned model would replace the $\log_2 W_{\mathrm{NR}}$ compilations of the bisection and allow per-region budgets inside one program.

REFERENCES

[1] B. Bichsel, M. Baader, T. Gehr, and M. Vechev, "Silq: A high-level quantum language with safe uncomputation and intuitive semantics," in *Proc. 41st ACM SIGPLAN Conf. Program. Lang. Design Implement. (PLDI)*, 2020, pp. 286–300.

[2] R. Seidel, S. Bock, R. Zander, M. Petrič, N. Steinmann, N. Tcholtchev, and M. Hauswirth, "Qrisp: A framework for compilable high-level programming of gate-based quantum computers," arXiv:2406.14792, 2024.

[3] K. Hirata and C. Heunen, "Qurts: Automatic quantum uncomputation by affine types with lifetime," *Proc. ACM Program. Lang.*, vol. 9, no. POPL, 2025.

[4] W. K. Wootters and W. H. Zurek, "A single quantum cannot be cloned," *Nature*, vol. 299, no. 5886, pp. 802–803, 1982.

[5] C. H. Bennett, "Logical reversibility of computation," *IBM J. Res. Develop.*, vol. 17, no. 6, pp. 525–532, 1973.

[6] F. Hua, Y. Jin, Y. Chen, S. Vittal, K. Krsulich, L. S. Bishop, J. Lapeyre, A. Javadi-Abhari, and E. Z. Zhang, "CaQR: A compiler-assisted approach for qubit reuse through dynamic circuit," in *Proc. 28th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS), Vol. 3*, 2023, pp. 59–71.

[7] M. DeCross, E. Chertkov, M. Kohagen, and M. Foss-Feig, "Qubit-reuse compilation with mid-circuit measurement and reset," *Phys. Rev. X*, vol. 13, no. 4, p. 041057, 2023.

[8] A. Paradis, B. Bichsel, S. Steffen, and M. Vechev, "Unqomp: Synthesizing uncomputation in quantum circuits," in *Proc. 42nd ACM SIGPLAN Conf. Program. Lang. Design Implement. (PLDI)*, 2021.

[9] A. Paradis, B. Bichsel, and M. Vechev, "Reqomp: Space-constrained uncomputation for quantum circuits," *Quantum*, vol. 8, p. 1258, 2024.

[10] Y. Ding, X.-C. Wu, A. Holmes, A. Wiseth, D. Franklin, M. Martonosi, and F. T. Chong, "SQUARE: Strategic quantum ancilla reuse for modular quantum programs via cost-effective uncomputation," in *Proc. ACM/IEEE 47th Annu. Int. Symp. Comput. Archit. (ISCA)*, 2020, pp. 570–583.

[11] G. Meuli, M. Soeken, M. Roetteler, N. Bjørner, and G. D. Micheli, "Reversible pebbling game for quantum memory management," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, 2019, pp. 288–291.

[12] C. Liu, L. Zhou, and B. Meng, "Quantum uncomputation of clean and dirty ancilla qubits," arXiv:2608.09578, 2026.

[13] D. Ittah, T. Häner, V. Kliuchnikov, and T. Hoefler, "QIRO: A static single assignment-based quantum program representation for optimization," *ACM Trans. Quantum Comput.*, vol. 3, no. 3, 2022.

[14] A. McCaskey and T. Nguyen, "A MLIR dialect for quantum assembly languages," in *Proc. IEEE Int. Conf. Quantum Comput. Eng. (QCE)*, 2021, pp. 255–264.

[15] M. Poletto and V. Sarkar, "Linear scan register allocation," *ACM Trans. Program. Lang. Syst.*, vol. 21, no. 5, pp. 895–913, 1999.